



## Low Complexity Learning-Based QTMTT Partitioning Scheme for Inter Coding in VVC Encoder

Ibrahim Taabane, Daniel Menard, Anass Mansouri, Sélima Sahraoui, Ali Ahaitouf

### ► To cite this version:

Ibrahim Taabane, Daniel Menard, Anass Mansouri, Sélima Sahraoui, Ali Ahaitouf. Low Complexity Learning-Based QTMTT Partitioning Scheme for Inter Coding in VVC Encoder. IEEE Access, 2024, 12, pp.141088-141103. <10.1109/access.2024.3469089>. <hal-04716699>

**HAL Id: hal-04716699**

**<https://hal.science/hal-04716699v1>**

Submitted on 6 Jun 2025

**HAL** is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY-NC-ND 4.0 - Attribution - Non-commercial use - No Derivative Works - International License

## RESEARCH ARTICLE

# Low Complexity Learning-Based QTMTT Partitioning Scheme for Inter Coding in VVC Encoder

**IBRAHIM TAABANE**<sup>1,2</sup>, **DANIEL MENARD**<sup>1</sup>, **ANASS MANSOURI**<sup>3</sup>,  
**SELIMA SAHRAOUI**<sup>1</sup>, **AND ALI AHAITOUF**<sup>2</sup>

<sup>1</sup>CNRS, IETR—UMR 6164, INSA Rennes, University of Rennes, 35000 Rennes, France

<sup>2</sup>Laboratory of Intelligent Systems, Geo-Resources and Renewable Energies, Faculty of Sciences and Technologies, Sidi Mohamed Ben Abdellah University, Fez 30000, Morocco

<sup>3</sup>Laboratory of Intelligent Systems, Geo-Resources and Renewable Energies, National School of Applied Sciences, Sidi Mohamed Ben Abdellah University, Fez 30000, Morocco

Corresponding author: Ibrahim Taabane (Ibrahim.Taabane@insa-rennes.fr)

This work was supported by the Campus France and Moroccan National Center for Scientific and Technical Research [Centre National pour la Recherche Scientifique et Technique (CNRST)] through the Hubert Curien Partnerships [Partenariat Hubert Curien (PHC)] Maghreb Project and conducted under the joint supervision of the Institute of Electronics and Digital Technologies (IETR), Rennes, France, and the Laboratory of Intelligent Systems, Geo-resources, and Renewable Energy (SIGER), Fez, Morocco.

**ABSTRACT** The Versatile Video Coding (VVC) standard, finalized in 2020 by the Joint Video Experts Team (JVET) and the Video Coding Experts Group (VCEG), marks a major advancement in video compression technology, offering a 50% efficiency improvement over its predecessor, the High-Efficiency Video Coding (HEVC) standard. A key innovation in the VVC standard is the Quad Tree with nested Multi-Type Tree (QTMTT) structure, essential for the partitioning process. However, this enhancement has led to increased coding complexity, posing challenges for real-time applications. To address this, our paper focuses on optimizing the partitioning process in the VVC encoder under the Random Access (RA) configuration. We propose a novel approach that leverages inter-prediction by integrating both coding and motion information across inter-frames to enhance coding efficiency. This solution is implemented on the Fraunhofer Versatile Video Encoder (VVenC). It utilizes a set of lightweight Light Gradient Boosting Machine (LightGBM) binary classifiers to accurately predict the optimal split mode for each Coding Unit (CU). Consequently, our approach significantly accelerates the VVenC encoding process. Experimental results show that our method reduces the runtime of the slower preset by 43.21%, with only a slight bitrate increase of 2.9%. These improvements not only significantly reduce computational complexity but also outperform several existing state-of-the-art methods.

**INDEX TERMS** Complexity reduction, compression efficiency, inter prediction, LightGBM, machine learning, versatile video coding (VVC).

## I. INTRODUCTION

With the popularity of high-resolution formats like 4K and 8K, video compression has become essential for efficiently storing and transmitting video content. According to a study by Statista, in the fourth quarter of 2023, online videos reached about 92% of internet users worldwide [1]. Additionally, the number of people watching live videos

is projected to reach 164.6 million by the end of this year [2]. As online video streaming and new applications like Virtual Reality (VR) and 360-degree video emerge, there is an increasing need to reduce bandwidth while maintaining high quality. Existing standards like High-Efficiency Video Coding (HEVC) [3] can no longer meet such high requirements. To address this, the Versatile Video Coding (VVC) standard [4] was developed by the Joint Video Experts Team (JVET), a collaboration between the ITU-T Video Coding Experts Group (VCEG) and the ISO/IEC Moving Picture

The associate editor coordinating the review of this manuscript and approving it for publication was Yun Zhang <sup>1</sup>.

Experts Group (MPEG). VVC was finalized in July 2020 and published in November 2020. It aims to significantly improve compression efficiency over HEVC. Thus, VVC claims to achieve up to a 50% reduction in bitrate compared to HEVC while maintaining the same subjective quality level [4]. This improvement is attributed to VVC's advanced techniques and algorithms that effectively reduce bitstream sizes, thereby ensuring superior video quality [5].

Among these advanced techniques is the new Coding Tree Unit (CTU) partitioning scheme, which introduces the Quad Tree with nested Multi-Type Tree (QTMTT) structure. In VVC, the size of the CTU has been increased from  $64 \times 64$ , as used in HEVC, to  $128 \times 128$ . Additionally, QTMTT supports more possible split modes, including binary and ternary trees, in both vertical and horizontal directions. This new partitioning technique significantly contributes to the increased complexity of the VVC encoder. According to a study in [6], this encoder dedicates about 97% of its encoding time to determine the optimal split mode. Another advancement in VVC is the introduction of new tools in the intra-prediction block, such as Intra Sub-Partition (ISP) [7], Multi Reference Line (MRL) [8], and Matrix-based Intra Prediction (MIP) [9]. These new tools increase the encoding time of the VVC encoder [6]. Many other advanced tools are related to inter-coding, including new inter-prediction modes such as Advanced Motion Vector Prediction (AMVP), merge, and skip modes [10]. Additionally, these modes are combined with Affine, Combined Intra-Inter Prediction (CIIP), and Geo, resulting in a total of 10 modes [10]. Moreover, VVC introduces the Multi-Transform Selection (MTS), which adds Discrete Sine Transform type 7 (DST-7) and Discrete Cosine Transform type 8 (DCT-8) to the existing DCT-2 [4]. VVC also introduces the Low-Frequency Non-Separable Transform (LFNST) which enhances coding efficiency by optimizing the handling of low-frequency energy between the transform and quantization steps [11].

These new tools also increase the VVC encoder runtime compared to HEVC, posing a challenge for practical applications. To address this challenge, various complexity reduction methods have been proposed to optimize VVC encoder performance without compromising quality. For instance, some methods use Convolutional Neural Networks (CNNs) [12], [13] or other fast algorithms [14], [15] to predict the optimal partitioning mode. Meanwhile, other methods focus on reducing the complexity of intra-mode selection or the MTS scheme through early termination or pruning techniques [16], [17]. Moreover, rate-distortion optimization using adaptive content modeling has also been explored to enhance VVC efficiency [18].

Nevertheless, the main limitations of some state-of-the-art methods for CU partitioning in VVC are their efficiency and performance. In particular, these methods are based on heuristic algorithms or deep learning (DL) models, which are computationally expensive and lack adaptability toward dynamic variations in video content characteristics. As a

result, they increase computational overhead and reduce compression efficiency. Indeed, DL models in the form of CNNs, which are used to predict the best partition, typically require significant computational resources and large datasets during both the training and inference stages [12]. To address data limitations, Generative Adversarial Networks (GANs) have been applied in many industrial areas, providing an effective data augmentation tool, as reported by Li et al. [19].

However, many studies in the literature use the VVC Test Model (VTM) [20] as a baseline for their approaches. While VTM is effective for benchmarking, it is not optimized for practical use. In contrast, this paper employs the Fraunhofer Versatile Video Encoder (VVenC) [21], developed by the Fraunhofer Heinrich Hertz Institute (HHI). VVenC is an optimized VVC implementation based on VTM software, offering similar performance with reduced runtime [21]. It targets real-time applications by enhancing various encoding tools. Despite these advancements, VVenC still faces challenges due to the inherent computational complexity of VVC, particularly in Coding Unit (CU) partitioning [6].

Consequently, VVC can lead to higher latency and greater computational demands [4]. This affects real-time video applications such as live streaming, video conferencing, and interactive gaming, where low latency and high responsiveness are crucial [22]. For example, in live streaming, lengthy encoding processes may lead to delays that disrupt the viewing experience. Similarly, in video conferencing, increased latency can impact communication fluidity [22]. To tackle this computational complexity issue, we aim to further optimize the VVenC encoder by addressing its partitioning block.

In this paper, our primary goal is to develop a solution better suited for real-time applications. We propose a machine learning (ML)-based approach using Light Gradient Boosting Machine (LightGBM) [23] binary classifiers to accelerate CU partitioning in inter-frames under the Random Access (RA) configuration. To achieve improved real-time performance, we selected the VVenC encoder over the commonly used VTM reference software [20], as VVenC offers a more optimized VVC implementation with reduced runtime [21]. Our approach enhances the CU partitioning process in the VVenC encoder and extends the evaluated presets to include “slower”, “slow”, and “medium”. Implemented on VVenC encoder version 1.7.0 and evaluated on several video sequences from the Common Test Condition (CTC) [24], our method significantly reduces encoding time with only a minimal increase in bitrate. As a result, this study sets a new benchmark for future research and paves the way for further optimizations, enhancing the VVenC encoder for real-time and practical applications.

The remaining sections of this paper are organized as follows: Section II reviews the partitioning structure in VVC and existing complexity reduction methods for VVC inter-coding. Section III outlines our proposed approach, covering features extraction, the classification algorithm structure, and

the training methodology. Section IV presents experimental results, including implementation details, performance evaluations, and comparisons with state-of-the-art techniques. Finally, the paper concludes with Section V.

## II. BACKGROUND AND RELATED WORKS

### A. PARTITIONING STRUCTURE IN VVC

Like most video encoders, the VVC standard includes the fundamental hybrid codec steps, encompassing partitioning, prediction, transform, quantization, and entropy coding [4]. These steps are illustrated in Figure 1. The partitioning step divides the video frame into smaller blocks for more efficient processing. Prediction includes intra-prediction, which estimates pixel values within the same frame, and inter-prediction, which uses data from previously encoded frames. The transform step converts spatial domain data into the frequency domain to concentrate signal energy. Quantization reduces the precision of these transformed coefficients to lower the bitrate while controlling quality. Finally, entropy coding compresses the quantized data using lossless techniques to reduce redundancy and achieve the final compression [4].

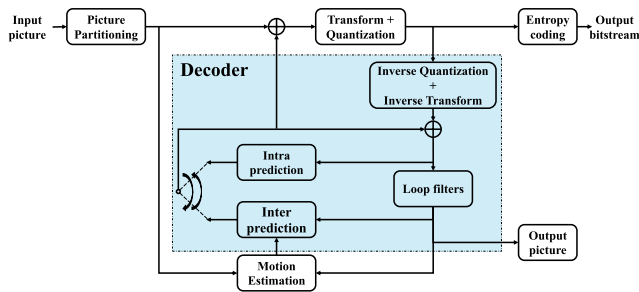


FIGURE 1. Block-based hybrid video coding framework.

In VVC, frames are initially divided into tile rows and columns to form CTU groups. Each CTU can handle either monochrome or three-color component video signals and consists of one or three Coding Tree Blocks (CTBs) [4]. Then, frames are further split into CTUs of up to  $128 \times 128$  pixels. These CTUs are then divided into CUs using a recursive Quad Tree (QT) structure along with the finer partition modes of the Multi-Type Tree (MTT), which includes Binary Tree (BT) and Ternary Tree (TT) splits. This results in six possible partition modes: No Split (NS), Quad Tree (QT), Binary Horizontal Tree (BH), Binary Vertical Tree (BV), Ternary Horizontal Tree (TH), and Ternary Vertical Tree (TV), as shown in Figure 2.

In addition, VVC uses the Rate-Distortion Optimization (RDO) process during encoding to select the best coding options for each CTU. This process evaluates various coding options for each CU to achieve the most balanced trade-off between bitrate and distortion. The objective is to minimize distortion while maintaining a low bitrate. VVC also imposes constraints on CU partition modes to improve efficiency, which limits potential split combinations

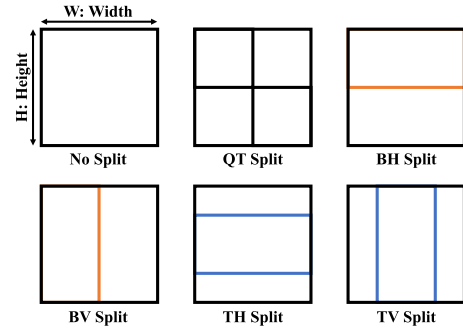


FIGURE 2. Visualization of QTMTT partition modes: QT (black), BT (orange), and TT (blue).

according to CU size. These constraints are outlined in Table 1, which shows the available split modes for each CU size.

TABLE 1. Available split modes for each CU size.

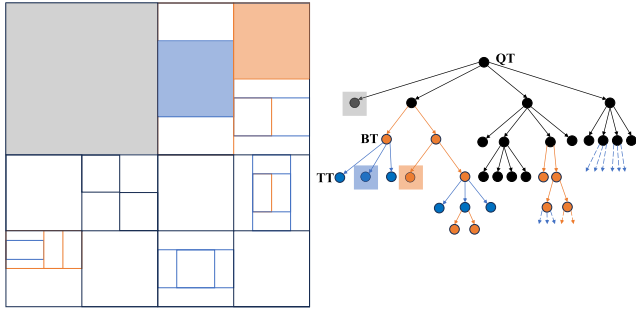
| Height | Width  |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|
|        | 128    | 64     | 32     | 16     | 8      | 4      |
| 128    | QT, BT | BT     | BT     | BT     | -      | -      |
| 64     | BT     | ALL    | BT, TT | BT, TT | BT, TH | BH, TH |
| 32     | BT     | BT, TT | ALL    | BT, TT | BT, TH | BH, TH |
| 16     | BT     | BT, TT | BT, TT | ALL    | BT, TH | BH, TH |
| 8      | -      | BT, TV | BT, TV | BT, TV | BT     | BH     |
| 4      | -      | BV, TV | BV, TV | BV, TV | BV     | -      |

These constraints prevent certain modes after specific divisions, ensuring effective CTU partitioning. The RDO process chooses the optimal split modes depending on the frame content and complexity. It identifies the partitioning scheme that results in the lowest Rate-Distortion (RD) cost, represented as  $RD_{cost}$  and calculated using Equation (1) below:

$$RD_{cost} = D + \lambda \cdot B \quad (1)$$

where  $D$  represents the distortion measured between the original and the predicted blocks, and  $B$  denotes the bitrate cost for a chosen mode, scaled by  $\lambda$ , the Lagrange multiplier. However, this process is computationally intensive and can be time-consuming. Additional factors such as the Quantization Parameter (QP), and the depth of QT and MTT also affect the encoding efficiency and time.

The illustration in Figure 3 shows an example of CTU partitioning using the QTMTT scheme, along with its corresponding partition tree. In this figure, the black nodes and arrows represent the QT partitions. Subsequently, the BT partition modes are represented by orange arrows and nodes, used in both horizontal and vertical orientations. Similarly, the blue arrows and nodes represent the TT partition modes, used in both directions as well. This visual representation with color-coding helps make the hierarchical partitioning



**FIGURE 3.** Illustration of an example of a CTU QTMTT partition scheme and its division tree: QT (black), BT (orange), TT (blue).

process in the VVC algorithm easier to understand. Moreover, the gray box around the leaf corresponds to the gray-filled CU, just like the orange and blue-filled CUs, to improve the visibility of each CU in the CTU.

This paper aims to enhance the VVenC encoder, which is an open-source version of the VTM encoder [21], offering improved compression efficiency compared to prior standards like HEVC. By utilizing advanced VVC techniques, VVenC operates at only 46% of the time required by the default reference software VTM [21]. Therefore, it is designed to deliver high quality with configurable presets that balance encoding speed and efficiency. These presets—faster, fast, medium, slow, and slower—adjust the encoder’s internal parameters to suit different use cases. One of the critical aspects of these presets is the configuration of the partitioning block, which directly impacts its performance [21]. The structure of this block in VVenC is governed by QT and MTT partitions, with different QT, BT, and TT depth values as shown in Table 2. In the slower and slow settings, the encoder maximizes partitioning depth with up to three levels of BT or TT splits, enabling the most detailed and efficient compression at the cost of increased runtime. Moving to the medium setting, the encoder permits up to two levels of BT or TT splits, balancing speed and compression efficiency. In the fast and faster settings, the encoder employs minimal partitioning complexity, using only QT to allow for rapid encoding with reduced compression efficiency [21].

**TABLE 2.** Maximum depth values for each split mode and encoding preset.

| Maximum Depth | Faster | Fast | Medium | Slow | Slower |
|---------------|--------|------|--------|------|--------|
| QT            | 4      | 4    | 4      | 4    | 4      |
| BT            | 0      | 0    | 1      | 2    | 3      |
| TT            | 0      | 0    | 1      | 2    | 3      |

Consequently, the CU partitioning introduces a high computational complexity to the encoder, especially in the inter-coding mode, where multiple reference frames and motion vectors (MVs) are involved. To speed up this process, several methods have been proposed. Section II-B reviews

some of these state-of-the-art methods and highlights their main contributions and limitations.

## B. PARTITIONING ACCELERATION APPROACHES FOR VVC INTER CODING

In recent years, various approaches have been developed to optimize the VVC standard, particularly its partitioning block in intra and inter-coding. These approaches can be broadly categorized into heuristic, ML, and DL techniques.

In [25], Tang et al. introduced a fast CTU partition decision algorithm for VVC intra and inter-coding. The algorithm uses a block-level Canny edge detector for intra-coding to determine the direction and strength of edges [35]. This allows the algorithm to skip certain partition modes selectively. In inter-coding, the algorithm prunes partitions using a custom metric derived from the pixel value differences between the current frame and its reference frames. The algorithm sets a threshold for this difference to determine whether to terminate the partitioning process early for less likely splits. This approach aims to reduce computational demands by avoiding unnecessary processing, achieving an Encoding Time Reduction (ETR) of 31% and a Bjøntegaard Delta Bitrate (BDBR) increase of 1.34%. In [32] Kuang et al. introduced a method to reduce partitioning complexity in both intra and inter-prediction for VVC. Their algorithm utilizes historical data from previously analyzed partitions to determine the most efficient partitioning path. It incorporates direction-based and position-based skips for BT and TT partitions, along with a depth-based skip for MTT partitions. This method decreases encoding time across various configurations, achieving average time savings of 42.47% in AI, 40.46% in RA, and 40.38% in Low Delay P (LDP) configurations, while maintaining acceptable increases in BDBR by 0.45%, 1.08%, and 1.18% in AI, RA and LDP respectively. In [33] authors proposed a method that utilizes coding information from neighboring CUs to predict the area of the current CU, determining the necessity of splitting modes. It also employs the temporally optimal coding mode generated during the prediction process to narrow down candidate modes, speeding up the coding process. Additionally, it exploits the distribution of neighboring prediction modes to assess the motion complexity of the current CU, enabling earlier skipping of unnecessary prediction modes. The approach has been implemented on the VTM 11.0, and it achieved a reduction in coding complexity by 40.08% on average, with an increase in BDBR.

In [14], the authors proposed an ML-based solution to address the increased encoding time resulting from the QT plus BT (QTBT) structure. By utilizing Random Forest (RF) classifiers, this method predicts the most probable partition modes for each coding block, thereby achieving a tunable balance between complexity reduction and coding loss. Implemented in the Joint Exploration Model (JEM) 7.0, as VVC was not yet finalized, the software demonstrated encoding complexity reductions of 30-70% on average, with

**TABLE 3.** Overview of techniques for VVC inter prediction optimization.

| Approach              | Software reference | Optimization technique | ETR (%)   | BDBR (%)  | Year of publication |
|-----------------------|--------------------|------------------------|-----------|-----------|---------------------|
| Tang et al. [25]      | VTM 4.0            | Heuristic              | 31        | 1.34      | 2019                |
| Amestoy et al. [14]   | JEM 7.0            | ML                     | 30-70     | 0.7-3     | 2020                |
| Yeo et al. [26]       | VTM 11.0           | DL                     | 11.53     | 1.01      | 2021                |
| Pan et al. [27]       | VTM 10.0           | DL                     | 30.63     | 3         | 2021                |
| Kulupana et al. [28]  | VTM 6.0            | ML                     | 32-41     | 0.82-1.33 | 2021                |
| Yue Li et al. [29]    | VTM 7.0            | ML                     | 23.19     | 0.97      | 2022                |
| Liu et al. [30]       | VTM 10.0           | DL                     | 17-30     | 0.37-1.18 | 2022                |
| Li Jingyi et al. [31] | VVenC 1.0          | ML                     | 15.27     | 1.87      | 2022                |
| Wei et al. [32]       | VTM 8.0            | Heuristic              | 40.46     | 1.08      | 2022                |
| Tissier et al. [12]   | VTM 7.0            | DL+ML                  | 31.3-51   | 1.65-3.79 | 2022                |
| Shang et al. [33]     | VTM 11.0           | Heuristic              | 40.08     | 1.56      | 2023                |
| Xie et al. [34]       | VVenC 1.0.0        | ML                     | 7.71      | 1.48      | 2023                |
| Liu et al. [13]       | VTM 13.0           | DL                     | 33.8-52.2 | 1.14-2.60 | 2023                |

a BDBR increase ranging from 0.7% to 3.0%. In [28] they proposed an approach that uses specialized Decision Trees (DTs) and two sets of RF classifiers to predict (QT/MTT) and (Horizontal/Vertical) split decisions. A specialized tree selection algorithm extracts the most accurate DTs from each forest based on an offline validation set. This method has been implemented on the VTM software, and it achieved an ETR of 32-41% and a BDBR increase of 0.82-1.33%. Li et al. in [29] employ a temporal prediction model to predict the optimal CU partition, achieving an ETR of 23.19% and a BDBR increase of 0.97%. This model uses data from previously coded frames and incorporates MV difference data to refine predictions. However, this approach can limit parallelism, as it relies on information from previously encoded frames, necessitating a sequential processing step that constrains simultaneous encoding operations. In their work [31], authors introduced a solution based on RF classifiers that focuses on temporal information and texture complexity of the CUs. The proposed method applies to CUs of various sizes ( $64 \times 64$ ,  $32 \times 32$ ,  $16 \times 16$ ) the RF models. These models trained offline, utilize the Gini coefficient [36] for decision-making at each node, efficiently handling multi-dimensional data. This approach reduced the encoding time by 15.27% in RA mode, with a BDBR increase of 1.87%. Another implementation in [34], proposed by Xie et al., introduces an RF-based early termination algorithm. This algorithm leverages features extracted from both the temporal co-located block and the spatially adjacent block of the current CU. The RF model predicts whether to early terminate the inter-prediction process of the current CU. When implemented on VVenC, the method achieves an average ETR of 7.71% in RA mode, with a BDBR increase of 1.48%.

Yeo et al. proposed a CNN-based fast split mode decision algorithm for VVC inter-prediction in their study [26]. This method utilizes a multi-level tree CNN architecture to classify the split mode of each CU by considering several factors, including the original and residual images of a CU, the Picture Order Count (POC), and the CU-level QP value. When implemented in VTM-11.0, this algorithm achieved an average complexity reduction of 11.53%, with an increase in

the BDBR of 1.01%. These results show that the time-saving is very low, which is unsuitable for real-time applications. In their study [30], Liu et al. introduced a lightweight CNN-based method that predicts partitioning grids at the CTU level, achieving an ETR of 17% to 30% with a BDBR increase of 0.37% to 1.18%. In their other work [13], they presented a U-Net-based CNN approach that predicts optimal partition paths using multi-scale motion vector fields and a partition pruning algorithm, further enhancing VVC encoding efficiency. Tissier et al. [12] proposed a method that combines DL and ML to reduce VVC encoder complexity in the RA configuration while minimizing BDBR loss. This approach leverages MVs calculated before partitioning. Initially, a CNN processes the current CTU along with two reference CTUs determined by the MVs with the lowest  $RD_{cost}$ . A multi-class classifier based on DT models then predicts split probabilities based on the CNN-derived features. The encoder tests the top-N predicted splits to balance complexity reduction and BDBR loss. In the first configuration, this method achieves about 31.8% complexity reduction with a 1.11% BDBR loss on the CTC. However, the use of CNNs adds significant complexity to the implementation, as the network's training and inference phases require substantial computational resources, potentially impacting real-time processing capabilities. Pan et al. have proposed a fast encoder algorithm to enhance implementation efficiency, as detailed in [27]. They introduced a multi-branch CNN that operates at the CU level and performs binary classification of "Partition" or "Non-partition" by fully exploiting the texture and motion activity characteristics.

In conclusion, while heuristic, ML, and DL techniques each contribute significantly to optimizing the VVC partitioning block, they have distinct limitations. Specifically, heuristic methods often lack generalization. On the other hand, ML approaches are more adaptable but heavily depend on high-quality training data and can be computationally demanding, with a risk of overfitting. Furthermore, DL techniques, although highly effective, necessitate large datasets and extensive computational resources, thereby complicating integration and interpretation. Therefore, balancing ETR and

BDBR is crucial for effective video compression across all these methods.

Table 3 summarizes techniques for optimizing CU partitioning in inter-coding, including heuristic, ML, and DL methods. It details the software versions used, ETR percentages, BDBR improvements, and publication years. This table shows that most of these studies have been conducted using the VTM encoder. However, there are two notable exceptions, as described by Li et al. [31] and Xie et al. [34], who employed the VVenC encoder. These studies focused on refining VVenC under RA configuration using the medium preset only. Building on this foundation, our research extends these optimizations to include the slow and slower presets in addition to the medium. It is important to note that the fast and faster in VVenC are restricted to using only the QT as previously described in Table 2, thereby obviating the need for further optimizations for these two presets. This expansion of focus underscores our contribution as a crucial effort in advancing the VVenC encoder towards practical real-time applications.

### III. PROPOSED APPROACH

In the field of video compression, the CU partitioning process significantly impacts both compression efficiency and encoding time. A study by Tissier et al. [6] demonstrated that optimizing this process could reduce computational complexity by over 97%. Building on this, our approach introduces a novel method to enhance CU partitioning within the VVC framework, specifically targeting the VVenC encoder. This method employs ML to accelerate CU partitioning by integrating features related to spatial and temporal complexities. These features include spatial metrics such as gradient values, pixel variance, and mean pixel values, which assess texture complexity and guide decisions on CU splitting. Temporal features are derived from motion vector

variances between consecutive frames, capturing motion dynamics essential for inter-frame coding. We used these features to train LightGBM models that predict the optimal split modes for each CU. These predictions are then utilized by the VVenC encoder to evaluate and select the best partition mode. As a result, our approach aims to optimize the encoding process by reducing its runtime while preserving compression efficiency.

The architecture of the proposed approach is shown in Figure 4, illustrating the enhancement of the partitioning block in the VVenC encoder. Initially, the inter-frame processing begins by extracting the Motion Vector Field (MVF) for  $4 \times 4$  pixel blocks within the frame, referencing the motion information from previous frames. This MVF is then used to derive the temporal features of the current CU. Additionally, texture features are extracted. Then, these extracted features are fed as input to the binary classifiers within the ML block. These classifiers predict the most probable split modes for the CU based on the extracted features. The predicted split modes are stored in a vector and subsequently evaluated by the encoder. The encoder tests these modes for the current CU, and upon processing the CU, it moves on to the next CUs. This process continues until the entire video is encoded into a bitstream file.

#### A. TEXTURE AND TEMPORAL FEATURES EXTRACTION

In video encoding, intra-frames correlate CU partition modes with texture complexity, while inter-frames introduce an additional layer of complexity because they depend on reference frames for prediction [4]. Our research enhances CU partitioning under RA configuration by integrating texture and temporal complexity, aiming to improve classification performance for optimal partitioning strategies. Furthermore, higher texture complexity generally indicates further splitting due to the close relation between the QTMTT structure and image texture. Whereas, image areas with more complex textures are usually further split. On the other hand, texture complexity can also reflect the direction selected for the partition modes [15]. When the complexity of texture in the horizontal direction is higher than in the vertical direction, CU will be more likely to be divided in the horizontal direction, and vice versa. Given these observations and their computational implications, a set of features was carefully selected to train binary classifiers. These features provide a comprehensive view of spatial and texture complexity, which is critically important for image quality. Additionally, temporal complexity describes the dynamics of a video and is essential for inter-coding mode. This method combines temporal and texture complexity features to improve prediction accuracy, which makes classifiers versatile and effective in handling a variety of video content.

#### 1) TEXTURE COMPLEXITY FEATURES

Since coding modes can vary with different QPs, our approach includes data from several QP values (22, 27, 32, and 37), which are commonly used in video coding standards

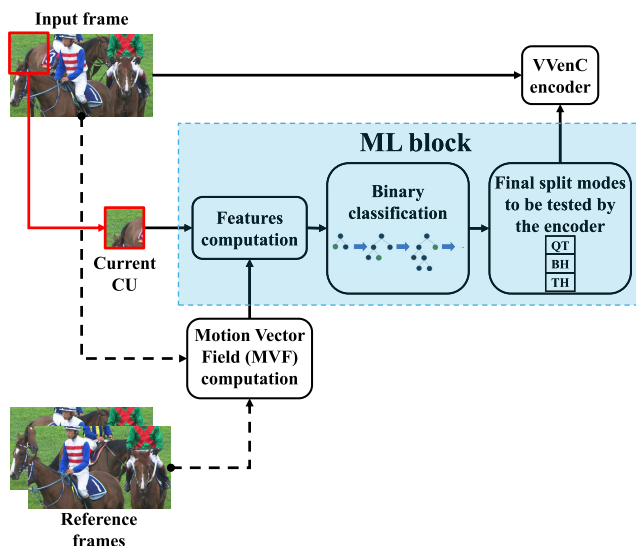
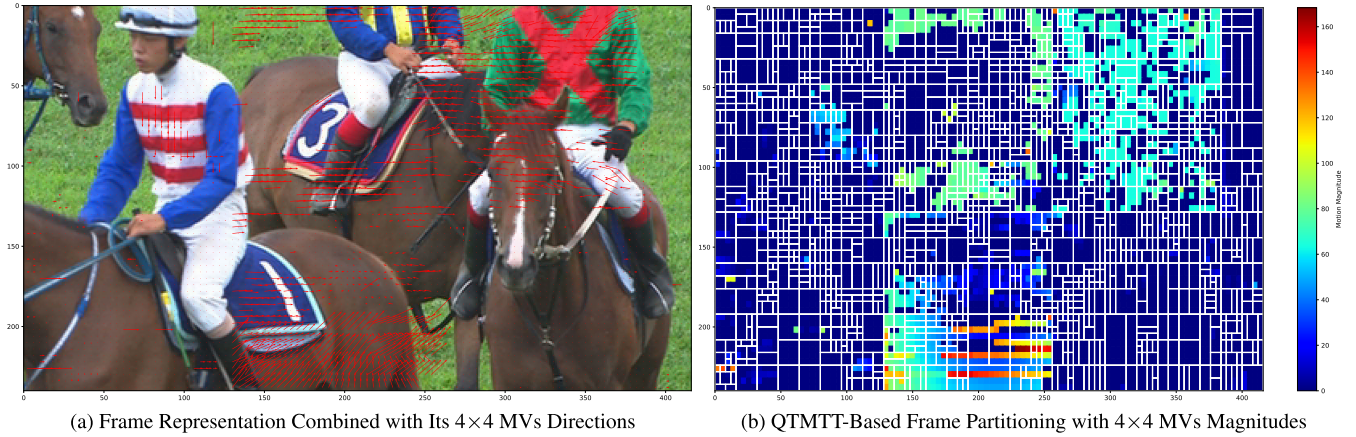


FIGURE 4. ML-based approach for optimizing CU partitioning in the VVenC encoder.



**FIGURE 5. Visualization of MVs in  $4 \times 4$  Blocks: MVs directions indicated by red arrows in (a) and MVs Magnitudes with CU partitioning structure in (b).**

like VVC [24]. The dataset was designed to capture the diversity of coding decisions across these QPs, enabling the models to handle CUs effectively under various quantization conditions. Additionally, to describe texture complexity, we calculate the following features using the pixel values denoted by  $I$ .

- **Quantization Parameter, QP:**  $QP \in \{22, 27, 32, 37\}$
- **Vertical gradient,  $\Delta_V$ :**

$$\Delta_V = \sum_{i=0}^{W-1} \sum_{j=0}^{H-1} |I(i, j) - I(i, j+1)| \quad (2)$$

- **Horizontal gradient,  $\Delta_H$ :**

$$\Delta_H = \sum_{i=0}^{W-1} \sum_{j=0}^{H-1} |I(i, j) - I(i+1, j)| \quad (3)$$

- **CU Pixels Mean,  $\mu_{CU}$ :**

$$\mu_{CU} = \frac{1}{W \cdot H} \sum_{i=1}^W \sum_{j=1}^H I(i, j) \quad (4)$$

- **CU Pixels Variance,  $\sigma_{CU}^2$ :**

$$\sigma_{CU}^2 = \frac{1}{W \cdot H} \sum_{i=1}^W \sum_{j=1}^H (I(i, j) - \mu_{CU})^2 \quad (5)$$

- **Sub-CU's Texture Complexity,  $SCTC$ :**

$$SCTC_s = \frac{1}{k} \sum_{i=1}^k (\sigma_i^2 - \overline{\sigma_s^2})^2 \quad (6)$$

where  $W$  and  $H$  denote the width and height of the CU or the corresponding sub-CU, respectively. The term  $\sigma_i^2$  represents the variance of the  $i$ -th sub-CU, calculated using Equation (5). Meanwhile,  $\overline{\sigma_s^2}$  refers to the average variance of the sub-CUs for a given split mode, denoted by  $s$ . The variable  $k$  indicates the number of sub-CU partitions, as detailed in Table 4:

**TABLE 4. Split modes with their corresponding Sub-CU parts number.**

| Split mode ( $s$ )              | QT | BH | BV | TH | TV |
|---------------------------------|----|----|----|----|----|
| Number of sub-CUs parts ( $k$ ) | 4  | 2  | 2  | 3  | 3  |

## 2) TEMPORAL COMPLEXITY FEATURES

Temporal complexity was assessed by selecting features that quantify changes between consecutive frames. These features rely on motion information, frame relations, and frame differences, which are crucial for inter-coding. These features were calculated using the MVF created by the MVs associated with each  $4 \times 4$  pixel block within the frame. These MVs are crucial in computing the variance within each CU and its corresponding sub-CUs. The process leverages the  $4 \times 4$  MVs that define the movement patterns within each block or sub-block. This method provides a detailed understanding of motion consistency and variability across different sections of the video frame. Figure 5a illustrates the MVs of each  $4 \times 4$  block within frame number 16 of the *RaceHorses* video sequence, captured at a resolution of  $416 \times 240$  pixels. Additionally, Figure 5b complements this by depicting the magnitude of each  $4 \times 4$  block in the same frame, employing the QTMTT scheme for CU partitioning. As illustrated in Figures 5a and 5b, dynamic elements within the frame exhibit higher magnitude values and more varied MV directions, necessitating increased CU partitioning levels. Conversely, static elements display stable motion patterns, resulting in fewer divisions.

- **Temporal Layer (Tid):** Indicates the temporal layer of a frame in a video sequence.
- **QT depth (QTD):** Represents the QT depth for a block, related to motion or texture complexity.
- **MTT depth (MTTD):** Similar to QTD, but for the MTT structure.
- **MVs Variance:** Each CU is subdivided into smaller blocks of  $4 \times 4$  pixels. The variance of MVs both in horizontal ( $MV_x$ ) and vertical ( $MV_y$ ) directions within these blocks provides significant insights into the motion characteristics across the CU. Given a CU of any size

with width  $W_{CU}$  and height  $H_{CU}$ , partitioned into  $N$   $4 \times 4$  blocks, where  $N$  is dependent on the CU dimensions and calculated as defined in Equation (7), the variance of the MVs is calculated as follows:

$$N = \frac{W_{CU} \times H_{CU}}{16} \quad (7)$$

$$\sigma_{MV_x}^2 = \frac{1}{N} \sum_{i=0}^{N-1} (MV_x(i) - \mu_{MV_x})^2 \quad (8)$$

$$\sigma_{MV_y}^2 = \frac{1}{N} \sum_{i=0}^{N-1} (MV_y(i) - \mu_{MV_y})^2 \quad (9)$$

where  $MV_x(i)$  and  $MV_y(i)$  are the horizontal and vertical components of the MV for the  $i$ -th  $4 \times 4$  block within the CU, respectively. The mean values,  $\mu_{MV_x}$  and  $\mu_{MV_y}$ , are computed as the average of the MV components across all blocks in the CU as described in Equation (10) and Equation (11):

$$\mu_{MV_x} = \frac{1}{N} \sum_{i=0}^{N-1} MV_x(i) \quad (10)$$

$$\mu_{MV_y} = \frac{1}{N} \sum_{i=0}^{N-1} MV_y(i) \quad (11)$$

### 3) FEATURES COMPUTATION

Features computations are performed for the entire CU and each of its sub-CU components, designated as  $S_i$ . For a square-shaped CU, the features for the sub-CUs are computed as illustrated in Figure 6a. In the scenario of a rectangular CU, the feature calculations are conducted by assessing its respective sub-CUs, as depicted in Figure 6b.

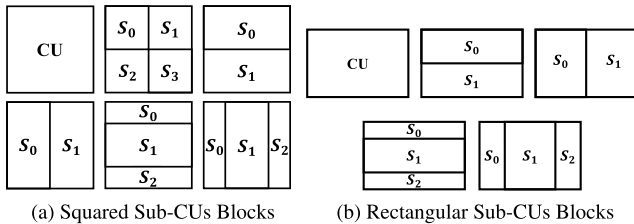


FIGURE 6. CU blocks and Sub-CU blocks for features extraction.

### B. CLASSIFICATION ALGORITHM STRUCTURE

Previous research demonstrates that early identification of whether a CU requires splitting, as well as the determination of the split direction (horizontal or vertical for BT and TT types), can significantly lower computational load [14], [15], [37]. These decisions lead to binary classification challenges: determining whether a split should occur and its direction.

Our approach employs LightGBM for classification, selected for its effectiveness in handling nonlinear classification scenarios [23]. Furthermore, instead of using a multiclassifier to classify the five classes (QT, BH, BV, TH, TV), five distinct binary classifiers were used, each predicting whether the current CU should be split according

### Algorithm 1 CU Partitioning Optimization Algorithm Using LightGBM Binary Classifiers

**Require:** CU

**Ensure:** Optimal split mode for the CU or termination of its encoding

```

1: Initialize binary classifiers for each split type:  $C_{QT}$ ,  $C_{BH}$ ,  $C_{BV}$ ,  $C_{TH}$ ,  $C_{TV}$ 
2:  $P_{QT} = P_{BH} = P_{BV} = P_{TH} = P_{TV} = 0$ 
3: if No partition mode is allowed then
4:   Terminate encoding for the current CU
5: return Encoding termination signal
6: end if
7: Initialize a list to hold split probabilities:  $L_P$ 
8: for Each CU in the CTU do
9:   F = Compute Feature (CU, W, H)
10:  if canSplit(QT) then
11:     $P_{QT} = C_{QT}(\mathbf{F})$ 
12:  end if
13:  if canSplit(BH) then
14:     $P_{BH} = C_{BH}(\mathbf{F})$ 
15:  end if
16:  if canSplit(BV) then
17:     $P_{BV} = C_{BV}(\mathbf{F})$ 
18:  end if
19:  if canSplit(TH) then
20:     $P_{TH} = C_{TH}(\mathbf{F})$ 
21:  end if
22:  if canSplit(TV) then
23:     $P_{TV} = C_{TV}(\mathbf{F})$ 
24:  end if
25:   $L_P.append([P_{QT}, P_{BH}, P_{BV}, P_{TH}, P_{TV}])$ 
26:  Sort  $L_P$  in descending order based on probabilities
27:  if  $\max(P)_{P \in L_P} < TH_{NS}$  then
28:    Terminate encoding of the current CU
29:  return Encoding termination signal
30: else
31:   Select the top-3 split modes with the highest probabilities
32:   Test these split modes on the current CU
33:   Select and apply the optimal mode on this CU
34: end if
35: Proceed to encode the next CU
36: end for

```

to its criteria. This segmentation enhances the separation between classes and thereby improves classification performance. Consequently, five binary classifiers were introduced:  $C_{QT}$ ,  $C_{BH}$ ,  $C_{BV}$ ,  $C_{TH}$ , and  $C_{TV}$ . Each classifier functions as an independent split examiner, where each classifier  $C_s$  determines whether to apply the split  $s$  or not. These classifiers play a critical role in the decision-making process by assessing the necessity of each potential split.

The proposed Algorithm 1 details our method for accelerating the CU partitioning in the RA coding mode.

By leveraging binary classifiers trained to predict the likelihood of various CU split modes, including QT, horizontal and vertical binary splits (BH and BV), and asymmetric horizontal and vertical ternary splits (TH and TV), the algorithm efficiently prioritizes the most promising split configurations for each CU. In this framework, Algorithm 1 initializes the classifiers by integrating them into the encoder. Subsequently, the first CU embarks on its split mode determination by feature extraction as described in section III-A. This includes assessing the size of each CU for its compatibility with certain modes before initiating the classification process. If the size criteria are met, the CU is evaluated by each eligible classifier to assess the probability of optimal representation by one of the five possible split modes. These classifiers are built to detect which division mode best balances encoding efficiency with computational complexity, quantifying this through a probability score denoted as  $P_{xx}$ . Once each classifier has evaluated the CU, the resulting probabilities are aggregated into a list, denoted  $L_P$ , in descending order. The algorithm then prioritizes split modes based on these probability scores. If the CU size permits, the algorithm selects the top-N modes with the highest probability scores as candidates for encoding. Our experiments indicate that selecting  $N=3$  achieves the best trade-off between bitrate increase and runtime reduction. However, if the CU size allows for fewer than three modes, all available modes are considered candidates. Subsequently, the encoder tests these candidates to identify the one that minimizes the  $RD_{cost}$ . Conversely, suppose all predicted probabilities fall below a predefined threshold, denoted as  $TH_{NS}$  and set to 0.25. In that case, the encoder determines that no further splitting is necessary for the current CU and proceeds to encode it accordingly. This iterative process continues until all CUs within the CTU have been processed. To address the potential misclassification issue and ensure robustness in the encoding process, additional checks on CU size and the number of modes selected as candidates have been implemented. This measure aims to guarantee that at least one of the selected modes represents the optimal split for a given CU. By including these checks, the algorithm introduces a dual selection mechanism, offering flexibility and resilience. This mechanism provides a secondary option, ensuring that even if the primary split mode is not accurately identified, a close alternative is available.

### C. TRAINING METHODOLOGY

Figure 7 depicts the workflow of the proposed method, which is comprised of three main stages: dataset creation, binary classifiers training, and the implementation and evaluation of the proposed solution.

Initially, a large dataset of training video sequences is used to extract features related to texture and temporal complexity. These features, along with their corresponding CU split modes, form the basis of our dataset. Due to its superior coding efficiency, the slower preset is used for extracting features during the encoding process [21]. Subsequently, this

dataset is used to train a set of binary classifiers. These classifiers are then integrated into the VVenC encoder to enhance its CU partitioning efficiency. Finally, to validate the effectiveness of the proposed method, we conducted evaluations using the slower, slow, and medium presets, with the slower serving as the benchmark for comparison. The evaluation results are detailed in section IV.

#### 1) DATASET CREATION

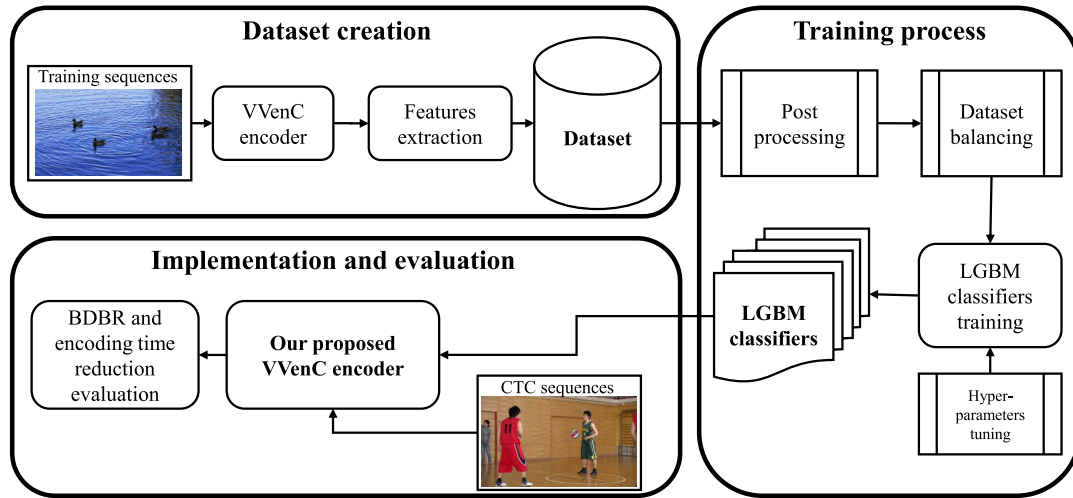
The dataset for our study was generated by encoding video sequences using the VVenC encoder with the “slower” preset under the RA configuration. This preset was selected because its encoding performance closely aligns with that of the VTM, as reported in [21]. By using this preset, we ensured that our dataset reflects an accurate representation of the VVC encoding characteristics. During this encoding process, we extracted features from CUs and labeled them based on their split modes, capturing all relevant textural and temporal characteristics. Each CU was assigned a label corresponding to its split mode, including QT, BH, BV, TH, TV, or NS. To train our five binary classifiers ( $C_{QT}$ ,  $C_{BH}$ ,  $C_{BV}$ ,  $C_{TH}$ , and  $C_{TV}$ ), we created separate datasets for each classifier. Each dataset consisted of instances belonging to the specific split mode and instances categorized as no split. This approach led to five binary classification problems, specifically: (QT vs. NS), (BH vs. NS), (BV vs. NS), (TH vs. NS), and (TV vs. NS).

To ensure that each dataset was balanced, we applied an undersampling technique [38], which equalized the number of samples for both classes (‘split’ and ‘no split’). This strategy aimed to improve the performance and robustness of the classifiers by preventing class imbalance.

**TABLE 5. Breakdown of CTU distribution across different resolution classes in the dataset.**

| Class | CTUs per frame | Encoded B-frames | CTUs per video | Number of videos | Total CTUs |
|-------|----------------|------------------|----------------|------------------|------------|
| A1    | 512            | 124              | 63488          | 4                | 253952     |
| A2    | 480            | 124              | 59520          | 17               | 1011840    |
| B     | 120            | 124              | 14880          | 9                | 133920     |
| C     | 18             | 124              | 2232           | 9                | 20088      |
| D     | 3              | 124              | 372            | 9                | 3348       |
| E     | 50             | 124              | 6200           | 4                | 24800      |
| Total | -              | -                | -              | 52               | 1447948    |

Table 5 details the final dataset, which consists of 1,447,948 CTUs distributed across different resolution classes: A1 (253,952 CTUs), A2 (1,011,840 CTUs), B (133,920 CTUs), C (20,088 CTUs), D (3,348 CTUs), and E (24,800 CTUs). Higher resolution classes, like A1 and A2, contain more CTUs because these videos typically have fewer subdivisions into smaller CUs, requiring a larger sample size to achieve sufficient CUs for analysis. Conversely, lower resolution classes, such as C and D, have fewer CTUs but are more frequently subdivided into smaller CUs. It is important to note that the video sequences used to create this dataset are



**FIGURE 7.** Proposed method Workflow: Dataset creation, training, and performance evaluation in the VVenC encoder.

different from the CTC sequences used for evaluation. This difference helps the binary classifiers better generalize their predictions on new video data.

## 2) BINARY CLASSIFIERS TRAINING

Light Gradient Boosting Machine (LightGBM), is an efficient gradient boosting framework designed for distributed and efficient learning, particularly useful in classification tasks. Developed by Microsoft, it stands out for its use of gradient-based one-sided sampling and exclusive feature bundling, which significantly reduce memory usage and increase the speed of computations [23]. LightGBM generally outperforms random forest (RF) and decision tree (DT) models in binary classification tasks due to its superior prediction accuracy, faster training, and quicker prediction times [39]. It is particularly suited for handling large datasets and performs well with categorical features as it uses a histogram-based algorithm that bins continuous features into discrete bins, thus speeding up the training process. It supports both binary and multiclass classification, making it a versatile tool in the ML practitioner's toolkit. Its ability to handle sparse data and its lower memory usage compared to other boosting algorithms make it an attractive choice for many real-world classification problems [23].

LightGBM models have many hyper-parameters that can be adjusted to enhance their classification performance. These include the maximum number of leaves per tree, learning rate, tree depth, and the minimum required samples at a leaf node. Additionally, other hyper-parameters dictate the proportion of samples and features utilized in training each tree [23]. Tuning these hyper-parameters is critical for optimizing the performance of LightGBM models. Techniques such as grid search and Bayesian optimization are commonly employed to ascertain the optimal settings for these parameters. This study utilizes the Tree-structured Parzen Estimator (TPE) algorithm [40], implemented through

the Optuna framework which is a versatile Python library that supports various optimization methods including random search and grid search [41]. Table 6 details the optimal hyperparameters values obtained.

**TABLE 6.** Training parameters for LightGBM models.

| Parameter Name       | Parameter Setting |
|----------------------|-------------------|
| Learning rate        | 0.05              |
| Maximum Depth        | 10                |
| Number of leaves     | 32                |
| Number of estimators | 96                |
| Boosting type        | GBDT              |
| Objective            | binary            |

The training process prioritizes balancing both the CU sizes and split modes to form a balanced dataset. This method enhances the effectiveness of training the LightGBM models and prevents classification issues related to data imbalance. The dataset was divided into three subsets: 70% for training, 15% for validation, and 15% for testing. These subsets are separate, with no overlap. The training set is used to train the model and help it learn from the data. The validation set is used to tune the model. The test set measures how well the model performs on new and unseen data.

Following training and validation, the classifiers were integrated into the VVenC encoder 1.7.0, as outlined in Algorithm 1, to predict the split mode for CUs during the encoding process. Finally, the evaluation results of this solution are detailed in the following section IV.

## IV. EXPERIMENTAL RESULTS

### A. IMPLEMENTATION DETAILS AND EVALUATION METRICS

The proposed algorithm was implemented in the slower, slow, and medium presets of the VVenC 1.7.0, and evaluated following the JVET common test conditions alongside the

specified configurations indicated in Table 7. The evaluation is performed under RA configuration, with a Group Of Pictures (GOP) size of 32, an I-frame period of 64, and QP values set to 22, 27, 32, and 37. This evaluation includes a variety of video sequences grouped into six different categories: A1 and A2 (4k), B (1080p), C (480p), D (240p), E (720p), and F (various resolutions).

**TABLE 7. Test and evaluation configurations of the proposed method.**

|                             |                    |
|-----------------------------|--------------------|
| Encoder                     | VVenC 1.7.0        |
| Coding Configuration        | Random Access (RA) |
| GOP size                    | 32                 |
| Intra-Period                | 64                 |
| Reference Preset            | Slower             |
| Quantization Parameter (QP) | 22, 27, 32, 37     |

To evaluate the proposed method, two widely used metrics in the video coding field are utilized. The first metric is the Bjøntegaard Delta Bitrate (BDBR), established by Gisle Bjøntegaard in [42]. This metric is used for comparing the bitrate efficiency between two different codecs or encoding strategies. It calculates the average difference in bitrate at the same quality level, typically measured using the Peak Signal-to-Noise Ratio (PSNR), as explained in [42]. To calculate BDBR, a weighted average is taken with a ratio of 6:1:1 for the luma component (Y) and the two chroma components (U and V) of a video signal, using Equation (12).

$$BDBR_{YUV} = \frac{(6 \cdot BDBR_Y + BDBR_U + BDBR_V)}{8} \quad (12)$$

The second metric is Encoding Time Reduction (ETR), which addresses the computational load of the encoding process. This metric is denoted by  $\Delta ETR$  and is calculated using Equation (13).

$$\Delta ETR = \frac{1}{4} \sum_{QP_i \in \{22, 27, 32, 37\}} \frac{T_R(QP_i) - T_P(QP_i)}{T_R(QP_i)} \quad (13)$$

This equation represents the average global time savings by comparing the encoding times of the original and proposed methods across four QP values (22, 27, 32, and 37).  $T_R$  and  $T_P$  denote the encoding times of the Reference and Proposed encoders, respectively.

These metrics are invaluable for comparing the effectiveness of new encoding algorithms or modifications to existing ones. Lower BDBR and encoding time values are desirable in scenarios with limited storage or bandwidth, such as streaming and mobile applications, as they indicate efficient resource use and suitability for real-time applications. Additionally, in consumer electronics and mobile devices, where processing power and battery life are limited, quicker encoding times can lead to significant energy savings and prolonged device usability. However, these two metrics often have an inverse relationship. Typically, strategies that reduce runtime might do so at the cost of increased bitrate

or reduced video quality. Whereas techniques that lower BDBR might require more complex calculations, thereby increasing computational complexity. This trade-off is a significant challenge in codec optimization. For instance, when employing a technique that limits the number of split modes evaluated by the encoder to reduce runtime, the optimal split mode may occasionally be excluded, resulting in an increase in BDBR. Conversely, testing more split modes can improve coding efficiency but at the cost of longer encoding times.

## B. CLASSIFICATION PERFORMANCE EVALUATION

To assess the performance of LightGBM binary classifiers, various metrics are commonly used. Each metric provides a different perspective on the classifier's performance, and together they offer a comprehensive understanding of its effectiveness. Some key metrics include:

- **Confusion Matrix:** Provides a detailed breakdown of the classifier's performance, including the number of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN).
- **Accuracy:** This metric measures the proportion of total correct predictions (both TP and TN) out of all predictions made.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (14)$$

- **Precision:** Precision is the ratio of TP predictions to the total positive predictions made. It indicates the classifier's exactness.

$$Precision = \frac{TP}{TP + FP} \quad (15)$$

- **Recall (Sensitivity):** Recall measures the classifier's ability to detect all relevant instances, highlighting its sensitivity.

$$Recall = \frac{TP}{TP + FN} \quad (16)$$

- **F1 Score:** The F1 Score is the harmonic mean of precision and recall, providing a balance between them.

$$F1 = \frac{2 \times TP}{2 \times TP + FP + FN} \quad (17)$$

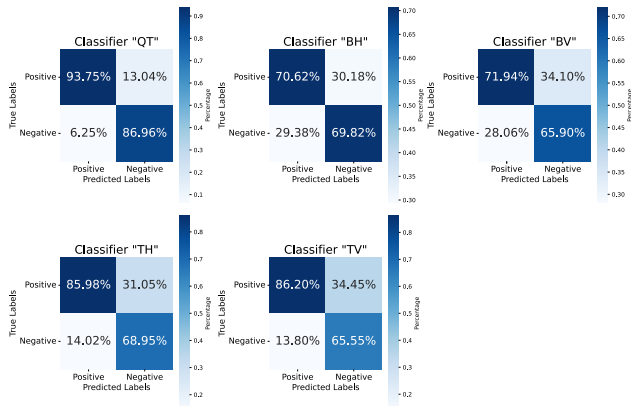
As a result, the accuracy, precision, recall, and F1 score of each classifier are summarized in Table 8. These classifiers were carefully trained using a combination of spatial and temporal features extracted from video sequences to determine if the CU should be split in one of five modes or left unsplit and encoded as it is. According to the assessment criteria, the QT classifier  $C_{QT}$  demonstrated outstanding performance, achieving an accuracy rate of 90.36% and an F1 score of 90.70%. This is followed by the binary and ternary classifiers, with the highest accuracy reaching 77.40% (for  $C_{TH}$ ). Additionally, upon comparing the performance of these classifiers with those employed by Amestoy in [14],

**TABLE 8. Performance evaluation results for each binary classifier.**

| Classifier | Accuracy (%) | Precision (%) | Recall (%) | F1 Score (%) |
|------------|--------------|---------------|------------|--------------|
| $C_{QT}$   | 90.36        | 87.85         | 93.75      | 90.70        |
| $C_{BH}$   | 70.22        | 70.10         | 70.62      | 70.36        |
| $C_{BV}$   | 68.93        | 67.97         | 71.94      | 69.90        |
| $C_{TH}$   | 77.40        | 73.19         | 85.98      | 79.07        |
| $C_{TV}$   | 75.80        | 71.15         | 86.20      | 77.95        |

it is evident that the proposed classifiers showed superior accuracy on average.

Furthermore, these classifiers were evaluated using confusion matrices, as illustrated in Figure 8. The  $C_{QT}$  classifier outperformed the others, accurately predicting whether a CU should be split using the QT mode. In contrast, the BT and TT classifiers exhibited some confusion between their split modes, due to their close relationship. To mitigate potential misclassifications, we adopted a strategy where the top three partition modes with the highest probabilities are always evaluated. This approach ensures that the three most likely split modes are consistently considered, thereby minimizing any loss in compression efficiency.


**FIGURE 8. Confusion matrices for each binary classifier.**

### C. PERFORMANCE EVALUATION OF THE PROPOSED VVenC ENCODER PRESETS

This section analyzes various presets (slower, slow, medium) over the CTC video classes (A1, A2, B, C, D, E, F). Table 9 illustrates the trade-offs between encoding efficiency, measured by BDBR, and encoding time reduction, measured by  $\Delta ETR$ . Results are shown for the CTC sequences from class A1 to class F. The table provides the average results for each class individually, as well as for all sequences combined from class A1 to F. Each preset corresponds to different encoding speeds and efficiency levels, providing an in-depth analysis of the performance range of the VVenC encoder.

The slower preset serves as the baseline for our comparison, as depicted in Figure 9. When comparing the slow preset to this baseline, it demonstrated a 5.78% increase in BDBR,

indicating reduced coding efficiency. However, this trade-off resulted in a significant reduction in encoding time, achieving up to a 74.38% decrease across all CTC classes. The primary goal of VVC development is to improve the quality of high-resolution videos [4]. Simplifying the encoding process for these high-resolution videos is crucial, as the time required for encoding grows with the resolution. classes A1 and A2 include 4K resolution sequences, while class B includes Full HD sequences.

Table 9 shows that for high-resolution classes A1, A2, and B, the slow preset exhibited bitrate increases of 4.30%, 7.33%, and 5.44%, respectively. Despite the increase in bitrate, the slow preset offered an overall encoding time reduction of 73.64%, while the proposed slow preset further improved this with a 79.02% reduction compared to the slower reference. When directly comparing the slow and proposed slow presets, our method optimized the slow preset by providing a time-saving of 20% across all CTC classes, with only a modest 1.71% increase in BDBR. This indicates a more efficient trade-off between coding efficiency and encoding time. The medium preset offers a higher BDBR of 12.32% and a runtime reduction of 91.72% across all classes, highlighting its prioritization of speed over efficiency.

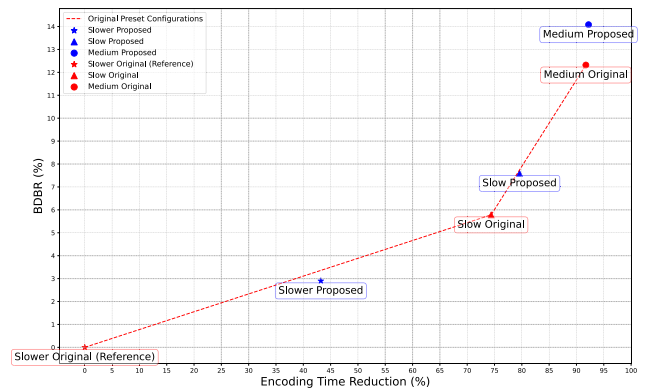
Table 9 clearly illustrates how class averages indicate the effectiveness of each preset. Slow presets balance speed and efficiency, suitable for applications requiring a compromise between these factors. In contrast, the medium presets prioritize speed, evident from their higher BDBR and faster encoding times. This analysis underscores the importance of selecting the appropriate preset based on application requirements, whether focusing on minimizing encoding time or maximizing compression efficiency. Table 9 also presents the BDBR and ETR of the proposed solution, reflecting performance across different scenarios. Compared to the original VVenC encoder, our method consistently achieves a better balance between reducing encoding time and maintaining a minimal increase in BDBR. For the slower preset, the proposed method consistently achieves significant improvements in ETR across all video classes. For instance, the average ETR improvement across class A1 videos is 40.09%, while class A2 and class B show enhancements of 44.09% and 44.62%, respectively. These improvements are crucial for applications requiring faster encoding times without compromising too much on quality. However, the BDBR values reveal a trade-off in bitrate efficiency. While the proposed method exhibits a slight increase in BDBR across all classes, the increases are relatively modest. For instance, the class A1 average BDBR is 2.09%, and for class A2, it is 2.84%. Class B, C, D, and E show BDBR increases of 2.74%, 3.36%, 3.18%, and 2.54%, respectively. The highest BDBR increase is observed in class F, with an average of 3.58%, suggesting a more significant impact on bitrate for certain types of content. These classes feature more complex motion and textures, demonstrating the preset's ability to handle such scenarios efficiently. Overall, the proposed slower preset achieves substantial time savings,

**TABLE 9.** Trade-offs between ETR(%) and BDBR(%): Performance evaluation of proposed VVenC presets vs. benchmark slower original preset.

| Preset        | Video               | Slower Proposed |                  | Slow Original |                  | Slow Proposed |                  | Medium Original |                  | Medium Proposed |                  |
|---------------|---------------------|-----------------|------------------|---------------|------------------|---------------|------------------|-----------------|------------------|-----------------|------------------|
|               |                     | BDBR (%)        | $\Delta$ ETR (%) | BDBR (%)      | $\Delta$ ETR (%) | BDBR (%)      | $\Delta$ ETR (%) | BDBR (%)        | $\Delta$ ETR (%) | BDBR (%)        | $\Delta$ ETR (%) |
| A1            | Campfire            | 2.18            | 50.60            | 3.10          | 70.70            | 4.35          | 77.45            | 8.14            | 91.11            | 9.45            | 92.14            |
|               | Tango2              | 2.59            | 36.13            | 5.01          | 73.38            | 6.53          | 78.66            | 11.56           | 91.41            | 13.21           | 91.86            |
|               | FoodMarket4         | 1.51            | 33.54            | 4.80          | 72.19            | 5.83          | 77.02            | 9.24            | 89.62            | 9.96            | 90.13            |
| Class average |                     | <b>2.09</b>     | <b>40.09</b>     | <b>4.30</b>   | <b>72.09</b>     | <b>5.57</b>   | <b>77.71</b>     | <b>9.65</b>     | <b>90.71</b>     | <b>10.87</b>    | <b>91.38</b>     |
| A2            | DaylightRoad2       | 3.58            | 39.43            | 8.71          | 73.40            | 11.29         | 78.87            | 16.79           | 91.88            | 19.49           | 92.30            |
|               | CatRobot1           | 3.17            | 39.98            | 9.13          | 73.32            | 11.58         | 79.38            | 16.38           | 91.83            | 18.74           | 92.26            |
|               | ParkRunning3        | 1.78            | 52.85            | 4.14          | 75.13            | 4.96          | 79.68            | 8.69            | 92.46            | 9.39            | 93.09            |
| Class average |                     | <b>2.84</b>     | <b>44.09</b>     | <b>7.33</b>   | <b>73.95</b>     | <b>9.28</b>   | <b>79.31</b>     | <b>13.95</b>    | <b>92.06</b>     | <b>15.87</b>    | <b>92.55</b>     |
| B             | MarketPlace         | 2.36            | 43.37            | 5.63          | 74.01            | 6.86          | 79.68            | 10.50           | 91.87            | 11.72           | 92.53            |
|               | RitualDance         | 2.67            | 44.25            | 4.77          | 72.29            | 6.26          | 78.76            | 11.08           | 91.51            | 12.68           | 92.19            |
|               | BQTerrace           | 2.78            | 41.28            | 5.54          | 76.68            | 7.22          | 80.91            | 11.10           | 92.25            | 12.79           | 92.66            |
|               | BasketBallDrive     | 3.19            | 47.95            | 5.80          | 75.45            | 7.78          | 80.06            | 13.70           | 92.84            | 15.85           | 93.37            |
|               | Cactus              | 2.71            | 46.25            | 5.46          | 76.01            | 7.01          | 80.88            | 11.48           | 92.37            | 13.03           | 93.05            |
| Class average |                     | <b>2.74</b>     | <b>44.62</b>     | <b>5.44</b>   | <b>74.89</b>     | <b>7.03</b>   | <b>80.06</b>     | <b>11.57</b>    | <b>92.17</b>     | <b>13.21</b>    | <b>92.76</b>     |
| C             | PartyScene          | 2.79            | 47.18            | 5.57          | 75.79            | 6.99          | 80.81            | 10.78           | 92.72            | 12.21           | 93.05            |
|               | BQMall              | 3.81            | 45.30            | 6.16          | 76.42            | 8.36          | 81.86            | 13.53           | 93.21            | 15.96           | 93.54            |
|               | BasketBallDrill     | 3.46            | 46.32            | 4.99          | 76.52            | 6.87          | 81.00            | 11.94           | 93.03            | 13.52           | 93.34            |
|               | RaceHorsesC         | 3.36            | 47.13            | 6.50          | 70.96            | 8.51          | 78.58            | 14.20           | 92.49            | 15.98           | 93.07            |
| Class average |                     | <b>3.36</b>     | <b>46.48</b>     | <b>5.81</b>   | <b>74.92</b>     | <b>7.68</b>   | <b>80.56</b>     | <b>12.61</b>    | <b>92.86</b>     | <b>14.42</b>    | <b>93.25</b>     |
| D             | BQSquare            | 2.79            | 32.61            | 5.99          | 80.55            | 7.57          | 84.15            | 11.25           | 92.69            | 12.51           | 92.97            |
|               | BlowingBubbles      | 2.84            | 44.94            | 5.09          | 78.99            | 6.53          | 83.40            | 10.77           | 93.59            | 12.32           | 93.68            |
|               | BasketBallPass      | 3.52            | 47.98            | 4.91          | 77.54            | 6.78          | 82.39            | 11.92           | 93.81            | 13.73           | 93.96            |
|               | RaceHorsesD         | 3.57            | 44.46            | 5.28          | 73.25            | 7.33          | 78.96            | 12.74           | 92.46            | 14.69           | 92.89            |
| Class average |                     | <b>3.18</b>     | <b>42.50</b>     | <b>5.32</b>   | <b>77.58</b>     | <b>7.05</b>   | <b>82.23</b>     | <b>11.67</b>    | <b>93.14</b>     | <b>13.31</b>    | <b>93.38</b>     |
| E             | FourPeople          | 2.90            | 42.04            | 4.36          | 77.72            | 6.22          | 81.59            | 9.50            | 92.06            | 11.41           | 92.25            |
|               | Johnny              | 2.22            | 31.98            | 4.56          | 77.91            | 6.18          | 81.05            | 9.81            | 90.73            | 11.49           | 90.76            |
|               | KristenAndSara      | 2.49            | 37.45            | 4.77          | 77.43            | 6.47          | 80.77            | 9.71            | 91.31            | 11.55           | 91.44            |
| Class average |                     | <b>2.54</b>     | <b>37.16</b>     | <b>4.56</b>   | <b>77.69</b>     | <b>6.29</b>   | <b>81.14</b>     | <b>9.67</b>     | <b>91.37</b>     | <b>11.48</b>    | <b>91.48</b>     |
| F             | ArenaOfValor        | 3.59            | 52.21            | 6.10          | 76.49            | 8.46          | 80.52            | 13.82           | 92.80            | 16.12           | 93.60            |
|               | SlideEditing        | 3.41            | 39.92            | 2.81          | 53.90            | 5.29          | 65.65            | 11.64           | 82.67            | 13.65           | 84.66            |
|               | BasketBallDrillText | 3.68            | 53.16            | 6.12          | 79.19            | 8.33          | 83.26            | 13.71           | 93.92            | 15.85           | 94.26            |
|               | SlideShow           | 3.63            | 44.81            | 15.68         | 68.62            | 18.97         | 74.73            | 29.35           | 89.42            | 31.79           | 90.14            |
| Class average |                     | <b>3.58</b>     | <b>47.53</b>     | <b>7.68</b>   | <b>69.55</b>     | <b>10.26</b>  | <b>76.04</b>     | <b>17.13</b>    | <b>89.70</b>     | <b>19.35</b>    | <b>90.67</b>     |
| Total Average |                     | <b>2.90</b>     | <b>43.21</b>     | <b>5.78</b>   | <b>74.38</b>     | <b>7.59</b>   | <b>79.58</b>     | <b>12.32</b>    | <b>91.72</b>     | <b>14.08</b>    | <b>92.21</b>     |

with a maximum runtime reduction of 53.16% with the *BasketBallDrillText* sequence and a minimum reduction of 31.98% with the *Johnny* sequence, averaging 43.21% across all sequences. This preset only marginally increases the BDBR, with a maximum increase of 3.81% for the *BQMall* sequence of class C, and a minimum increase of 1.51% for the *FoodMarket4* sequence, resulting in an overall average BDBR increase of just 2.9%. Thus, the proposed slower preset optimizes efficiency by significantly reducing encoding time while maintaining excellent compression performance.

Figure 9 illustrates that the proposed slower preset enhances performance by decreasing reference encoding time without compromising compression efficiency. The modest increase in BDBR suggests minimal impact on quality, underscoring the effectiveness of this approach in delivering both efficient and rapid video encoding. The effectiveness of our method in handling complex motion patterns is especially noticeable in fast-moving scenes like *ArenaOfValor* and

**FIGURE 9.** Total average BDBR vs. ETR for original and proposed VVenC presets.

*SlideShow* of class F, showing a reduction in encoding time of over 52%. In Figure 9, the blue star representing the proposed slower preset is well positioned compared to the red symbols

**TABLE 10.** Comparative analysis of trade-offs between ETR(%) and BDBR(%): Evaluating the performance of the proposed approach and some state-of-the-art methods.

| Class                | Yue Li [29] |                  | Tissier [12] |                  | Jingyi Li [31] |                  | Proposed Slower |                  |
|----------------------|-------------|------------------|--------------|------------------|----------------|------------------|-----------------|------------------|
|                      | BDBR (%)    | $\Delta$ ETR (%) | BDBR (%)     | $\Delta$ ETR (%) | BDBR (%)       | $\Delta$ ETR (%) | BDBR (%)        | $\Delta$ ETR (%) |
| A1                   | 1.05        | 28.08            | 1.81         | 51.1             | 1.59           | 16.61            | 2.09            | 40.09            |
| A2                   | 0.92        | 22.88            | 1.86         | 44.6             | 2.29           | 15.36            | 2.84            | 44.09            |
| B                    | 1.01        | 26.16            | 2.21         | 46.5             | 1.80           | 14.04            | 2.74            | 44.62            |
| C                    | 1.30        | 28.56            | 3.20         | 43.1             | -              | -                | 3.36            | 46.48            |
| D                    | 1.01        | 20.50            | 3.02         | 36.8             | -              | -                | 3.18            | 42.50            |
| E                    | 0.36        | 10.06            | 1.45         | 38.7             | -              | -                | 2.54            | 37.16            |
| F                    | -           | -                | 2.96         | 40.2             | -              | -                | 3.58            | 47.53            |
| <b>Total average</b> | <b>0.97</b> | <b>23.19</b>     | <b>2.36</b>  | <b>43.0</b>      | <b>1.87</b>    | <b>15.27</b>     | <b>2.90</b>     | <b>43.21</b>     |

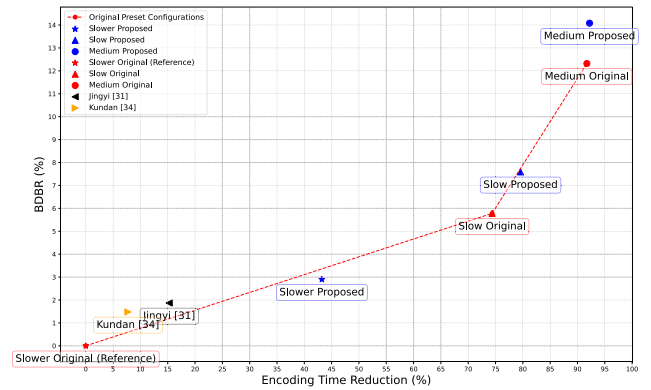
representing the original VVenC presets, indicating a better balance between ETR and BDBR. However, the proposed slow and medium presets are less efficient due to the MTT depths of each preset, as shown in Table 2, making these presets less optimized compared to the slower ones.

Overall, these findings validate the effectiveness of this approach, enabling more efficient VVC applications, particularly in real-time streaming and broadcasting where encoding speed is crucial.

#### D. COMPARISON WITH THE STATE-OF-THE-ART TECHNIQUES

In this section, we compare our solution with some state-of-the-art techniques. Most methods proposed in the literature were implemented on the VTM encoder, as detailed in Table 3. Table 10 presents a comparative analysis of the trade-offs between ETR and BDBR across various video sequence resolutions and categories. Our method utilizes the “slower proposed” setting, which closely aligns with the VTM and employs all its performance tools [21]. This comparison benchmarks against approaches by Li et al. [29], Tissier et al. [12], and Li et al. [31]. Our method achieves an average time-saving of 43.21%, outperforming Yue Li’s (23.19%) and Jingyi’s (15.27%), and slightly exceeding Tissier’s (43%). When compared to Yue Li’s approach, which employs a temporal prediction model, our method offers approximately double the time-saving at the cost of increased bitrate. Furthermore, our results are comparable to Tissier’s, which uses CNN and VTM, demonstrating the effectiveness of our method. It is worth noting that achieving efficiency in VVenC is more challenging than in VTM, as VVenC is already optimized with various heuristics to reduce encoding time [21]. Moreover, our method consistently delivers the best trade-offs, particularly for more complex categories (C, D, and F), resulting in significant encoding time reductions (44.62%, 42.50%, and 47.53%, respectively) with a slight increase in BDBR. Notably, for class F sequences across various resolutions, our method surpasses others by achieving greater time-saving while maintaining a moderate 3.58% increase in BDBR.

To ensure a fair comparison, our approach has been compared to notable studies using VVenC, specifically


**FIGURE 10.** Performance comparison: Original and proposed presets compared to some state-of-the-art methods implemented on VVenC.

those by Li et al. [31] and Xie et al. [34]. These studies report reductions in encoding time of 15.27% and 7.71%, respectively, with bitrate increases of 1.87% and 1.48%. Compared to these results, our approach achieves a significantly greater ETR of 43.21% across all CTC classes, with a reasonable BDBR increase of 2.9%. For high-resolution classes, we achieved an ETR of 42.93% with a BDBR increase of 2.55%. Figure 10 illustrates that these state-of-the-art methods do not achieve more than a 16% reduction in encoding time. In contrast, our approach offers significant time savings with only a slight increase in bitrate. Notably, there is a lack of studies focused on optimizing the VVenC encoder, with only the aforementioned two studies found.

These results underscore the effectiveness of our approach in balancing computational complexity with compression efficiency, surpassing other techniques to achieve a superior trade-off essential for practical encoding scenarios.

#### V. CONCLUSION

This paper presented a novel optimization approach aiming at reducing the complexity associated with the partitioning process. Specifically, the focus is on optimizing the VVenC encoder under the RA configuration to predict the partition mode of CUs using coding and motion information across inter-frames. The approach bypasses the standard RDO

process of VVC by employing LightGBM binary classifiers to predict split modes, thereby accelerating the encoding process with a minimal increase in bitrate. Experimental results confirm the effectiveness of the proposed solution, demonstrating a significant reduction in runtime by 43.21%, with only a slight increase in BDBR of 2.9%. This highlights how our method reduces computational complexity in VVC encoding and performs better than some current state-of-the-art techniques. Our study is a valuable resource for the video coding field, helping to improve the implementation of the VVC standard in real-time applications.

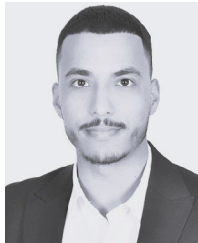
## ACKNOWLEDGMENT

The authors are thankful to INSA Rennes and USMBA University for their invaluable support and for granting them access to their resources.

## REFERENCES

- [1] Statista. (2021). *Leading Video Content Types Worldwide in 2021, By Share of Internet Users Who Watch Them*. [Online]. Available: <https://www.statista.com/statistics/1254810/top-video-content-type-by-global-reach/>
- [2] Statista. (2018). *Live Video Viewership in the United States From 2018 To 2022*. [Online]. Available: <https://www.statista.com/statistics/1284059/usa-live-video-viewership/>
- [3] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the high efficiency video coding (HEVC) standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.
- [4] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm, "Overview of the versatile video coding (VVC) standard and its applications," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 10, pp. 3736–3764, Oct. 2021.
- [5] Bianca. (2023). *The State of Video Codecs in 2023: A Comprehensive Overview*, 2023. Accessed: Dec. 15, 2023. [Online]. Available: <https://www.coconut.co/articles/the-state-of-video-codecs-in-2023-a-comprehensive-overview>
- [6] A. Tissier, A. Mercat, T. Amestoy, W. Hamidouche, J. Vanne, and D. Menard, "Complexity reduction opportunities in the future VVC intra encoder," in *Proc. IEEE 21st Int. Workshop Multimedia Signal Process. (MMSP)*, Sep. 2019, pp. 1–6.
- [7] S. De-Luxán-Hernández, V. George, J. Ma, T. Nguyen, H. Schwarz, D. Marpe, and T. Wiegand, "An intra subpartition coding mode for VVC," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2019, pp. 1203–1207.
- [8] J. Li, B. Li, J. Xu, and R. Xiong, "Intra prediction using multiple reference lines for video coding," in *Proc. Data Compress. Conf. (DCC)*, Apr. 2017, pp. 221–230.
- [9] M. Schäfer, B. Stallenberger, J. Pfaff, P. Helle, H. Schwarz, D. Marpe, and T. Wiegand, "Efficient fixed-point implementation of matrix-based intra prediction," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2020, pp. 3364–3368.
- [10] Y. Liu, M. Abdoli, T. Guionnet, C. Guillemot, and A. Roumy, "Statistical analysis of inter coding in VVC test model (VTM)," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2022, pp. 3456–3459.
- [11] M. Koo, M. Salehifar, J. Lim, and S.-H. Kim, "Low frequency non-separable transform (LFNST)," in *Proc. Picture Coding Symp. (PCS)*, Nov. 2019, pp. 1–5.
- [12] A. Tissier, W. Hamidouche, J. Vanne, and D. Menard, "Machine learning based efficient QT-MTT partitioning for VVC inter coding," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2022, pp. 1401–1405.
- [13] Y. Liu, M. Riviere, T. Guionnet, A. Roumy, and C. Guillemot, "CNN-based prediction of partition path for VVC fast inter partitioning using motion fields," 2023, *arXiv:2310.13838*.
- [14] T. Amestoy, A. Mercat, W. Hamidouche, D. Menard, and C. Bergeron, "Tunable VVC frame partitioning based on lightweight machine learning," *IEEE Trans. Image Process.*, vol. 29, pp. 1313–1328, 2020.
- [15] I. Taabane, D. Menard, A. Mansouri, and A. Ahaitouf, "Machine learning based fast QTMTT partitioning strategy for VVenC encoder in intra coding," *Electronics*, vol. 12, no. 6, p. 1338, Mar. 2023.
- [16] Q. Zhang, Y. Wang, L. Huang, and B. Jiang, "Fast CU partition and intra mode decision method for H.266/VVC," *IEEE Access*, vol. 8, pp. 117539–117550, 2020.
- [17] H. Yang, L. Shen, X. Dong, Q. Ding, P. An, and G. Jiang, "Low-complexity CTU partition structure decision and fast intra mode decision for versatile video coding," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 6, pp. 1668–1682, Jun. 2020.
- [18] Y. Chen, S. Wang, H. Ip, and S. Kwong, "Rate distortion optimization with adaptive content modeling for random-access versatile video coding," *Inf. Sci.*, vol. 645, Oct. 2023, Art. no. 119325.
- [19] Q. Li, Y. Tang, and L. Chu, "Generative adversarial networks for prognostic and health management of industrial systems: A review," *Expert Syst. Appl.*, vol. 253, Nov. 2024, Art. no. 124341.
- [20] VVC Test Model (VTM). Accessed: Jul. 12, 2024. [Online]. Available: [https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware\\_VTM](https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM)
- [21] (2020). VVenC Encoder. Accessed: Jul. 12, 2024. [Online]. Available: <https://github.com/fraunhoferhhi/vvenc/wiki>
- [22] (2024). *The Importance of Low Latency in Video Streaming*. Accessed: Aug. 25, 2024. [Online]. Available: <https://www.theoplayer.com/blog/the-importance-of-low-latency-in-video-streaming>
- [23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–20.
- [24] F. Bossen, J. Boyce, X. Suehring, X. Li, and V. Seregin, "JVET common test conditions and software reference configurations for SDR video," Joint Video Experts Team (JVET), Geneva, Switzerland, Tech. Rep. JVET-N1010, 2019.
- [25] N. Tang, J. Cao, F. Liang, J. Wang, H. Liu, X. Wang, and X. Du, "Fast CTU partition decision algorithm for VVC intra and inter coding," in *Proc. IEEE Asia-Pacific Conf. Circuits Syst. (APCCAS)*, Nov. 2019, pp. 361–364.
- [26] W.-H. Yeo and B.-G. Kim, "CNN-based fast split mode decision algorithm for versatile video coding (VVC) inter prediction," *J. Multimedia Inf. Syst.*, vol. 8, no. 3, pp. 147–158, Sep. 2021.
- [27] Z. Pan, P. Zhang, B. Peng, N. Ling, and J. Lei, "A CNN-based fast inter coding method for VVC," *IEEE Signal Process. Lett.*, vol. 28, pp. 1260–1264, 2021.
- [28] G. Kulupana, V. P. Kumar M, and S. Blasi, "Fast versatile video coding using specialised decision trees," in *Proc. Picture Coding Symp. (PCS)*, Jun. 2021, pp. 1–5.
- [29] Y. Li, F. Luo, and Y. Zhu, "Temporal prediction model-based fast inter CU partition for versatile video coding," *Sensors*, vol. 22, no. 20, p. 7741, Oct. 2022.
- [30] Y. Liu, M. Abdoli, T. Guionnet, C. Guillemot, and A. Roumy, "Light-weight CNN-based VVC inter partitioning acceleration," in *Proc. IEEE 14th Image, Video, Multidimensional Signal Process. Workshop (IVMSP)*, Jun. 2022, pp. 1–5.
- [31] J. Li, S. Zhang, and F. Yang, "Random forest accelerated CU partition for inter prediction in H.266/VVC," in *Proc. IEEE Int. Conf. Multimedia Expo. (ICME)*, Jul. 2022, pp. 01–06.
- [32] W. Kuang, X. Li, X. Zhao, and S. Liu, "Unified fast partitioning algorithm for intra and inter predictions in versatile video coding," in *Proc. Picture Coding Symp. (PCS)*, Dec. 2022, pp. 271–275.
- [33] X. Shang, G. Li, X. Zhao, and Y. Zuo, "Low complexity inter coding scheme for versatile video coding (VVC)," *J. Vis. Commun. Image Represent.*, vol. 90, Feb. 2023, Art. no. 103683.
- [34] K. Xie, J. Zhou, S. Zhang, and F. Yang, "Fast inter prediction mode decision method based on random forest for H.266/VVC," in *Proc. IEEE Int. Conf. Vis. Commun. Image Process. (VCIP)*, Dec. 2022, pp. 1–5.
- [35] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vols. PAMI-8, no. 6, pp. 679–698, Nov. 1986.
- [36] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, Oct. 2001.
- [37] Mario Saldanha, Gustavo Sanchez, Cesar Marcon, and Luciano Agostini, "Configurable fast block partitioning for vvc intra coding using light gradient boosting machine," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 3947–3960, May 2022.
- [38] (2023). *Handling Imbalanced Dataset With Undersampling Technique: Its Pros and Cons*, 2023. Accessed: Aug. 27, 2024. [Online]. Available: <https://medium.com/analytics-vidhya/handling-imbalanced-dataset-with-undersamplingtechnique-its-pros-and-cons-2c634e763a9e>
- [39] (2021). *Lightgbm: A Highly-Efficient Gradient Boosting Decision Tree*, 2021. Accessed: Apr. 30, 2024. [Online]. Available: <https://www.kdnuggets.com/2020/06/lightgbm-gradient-boosting-decision-tree.html>

- [40] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyperparameter optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2011, pp. 1–29.
- [41] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, vol. 18, Jul. 2019, pp. 2623–2631.
- [42] J. Ström, K. Andersson, R. Sjöberg, A. Segall, F. Bossen, G. Sullivan, J. R. Ohm, and A. Tourapis, *Working Practices Using Objective Metrics for Evaluation of Video Coding Efficiency Experiments*, Standard ISO/IEC TR 23002-8:2021, 2021.



(SIGER), USMBA. His research interests include video coding, efficient real-time and parallel architectures for the next-generation video coding standards, and hardware implementations.



of collaborative projects, he has been involved in different national and European projects. He has published more than 100 scientific papers in international journals and conferences. His research interests include low-power video codecs, approximate computing, and energy consumption. He is currently a member of different technical program committees of international conferences, such as ICASSP, SiPS, and DATE. Since 2018, he has been an Elected Member of the Technical Committee ASPS of the IEEE Signal Processing Society.



(SIGER) and the Head of the Team Embedded Systems, Electronics, and Telecommunication.



**SELIMA SAHRAOUI** received the Ph.D. degree in electronics and telecommunications from Polytechnic University Hauts-de-France (UPHF), in 2019. Since 2019, she has been a Lecturer of programming and digital electronics with INSA Rennes. In 2023, she began conducting research at the IETR Laboratory. Her research interests include video compression, real-time video coding, energy consumption in online video, and system-level energy consumption.



He supervised a dozen Ph.D. theses and published over 15 articles in renowned journals. His research interests include microelectronics and solar compounds, digital and analog design of integrated circuits, and image and data compression.

...